

Fold Explanations

Sophia Suckfüll

IT University Copenhagen

22. 09. 2026

This lecture

- ▶ Not directly exam relevant
- ▶ To help you understand a complicated concept
- ▶ Understanding is helpful for the exam!

On normal lists

Take the list

`[7, 5, 8, 9]`

If we do a `foldRight` with $\oplus$ with start element 0:

$(7 \oplus (5 \oplus (8 \oplus (9 \oplus 0))))$

On normal lists

Take the list

[7, 5, 8, 9]

If we do a `foldRight` with $\oplus$ with start element 0:

$$7 \oplus 5 \oplus 8 \oplus 9 \oplus 0$$

So order of operations

$$9 \oplus 0 \rightarrow 8 \oplus \rightarrow 5 \oplus \rightarrow 7 \oplus$$

On normal lists

Take the list

`[7, 5, 8, 9]`

If we do a `foldRight` with $\oplus$ with start element 0:

$$7 \oplus 5 \oplus 8 \oplus 9 \oplus 0$$

So order of operations

$$9 \oplus 0 \rightarrow 8 \oplus \rightarrow 5 \oplus \rightarrow 7 \oplus$$

So impossible on infinite lists... right?

What foldRight actually is

Take the list

[7, 5, 8, 9]

If we do a foldRight with f with start element 0:

$$\text{foldRight}([7, 5, 8, 9], 0, f) = f(7, \text{foldRight}([5, 8, 9], 0, f))$$

For lazy lists

The signature of f has to be $f(a:A, b:\Rightarrow B)$

For lazy lists

The signature of f has to be $f(a:A, b:\Rightarrow B)$

So b is an expression that is only evaluated when accessed!

For lazy lists

If $b \equiv \text{foldRight}(\text{tail}, \dots)$, then accessing b means continuing!
Conversely, *not* accessing b means stopping the fold!

Back to the example

[7, 5, 8, 9]

If we do a `foldRight` with f with start element 0:

$$\text{foldRight}([7, 5, 8, 9], 0, f) = f(7, \text{foldRight}([5, 8, 9], 0, f))$$

Back to the example

[7, 5, 8, 9]

If we do a `foldRight` with f with start element 0:

$$\text{foldRight}([7, 5, 8, 9], 0, f) = f(7, \text{foldRight}([5, 8, 9], 0, f))$$

Define f as

```
if a == 7  
then a  
else b
```

Back to the example

`[7, 5, 8, 9]`

If we do a `foldRight` with `f` with start element 0:

`foldRight([7, 5, 8, 9], 0, f) = f(7, foldRight([5, 8, 9], 0, f))`

Define `f` as

`if a == 7 then a else b`

And insert (because of call-by-name, we can simply paste the expression!):

```
val a = 7
if a == 7
then a
else foldRight([5, 8, 9], 0, f)
```

Back to the example

[7, 5, 8, 9]

```
val a = 7
if a == 7
then a
else foldRight([5,8,9], 0, f)
```

We have simple recursion that can stop :D

Back to the example

`[7, 5, 8, 9]`

`foldRight([7, 5, 8, 9], 0, f) = f(7, foldRight([5, 8, 9], 0, f))`

This makes sense, because the right argument is only evaluated when needed

For infinite lists

`[7, 1, 2, 3, 4, 5, ...]`

If we do a `foldRight` with f with start element 0:

$$\begin{aligned} & \text{foldRight}([7, 1, 2, 3, 4, 5, \dots], 0, f) \\ &= f(7, \text{foldRight}([1, 2, 3, 4, 5, \dots], 0, f)) \end{aligned}$$

Define f as `if a == 7 then a else b`

And insert (paste the expression!):

```
val a = 7
if a == 7 then a
else foldRight([1,2,3,4,5,...], 0, f)
```

No problem!

What about foldLeft?

Take again

[7, 5, 8, 9]

`foldLeft([7,5,8,9], 0, f)` *must* end up as

$f(f(f(f(0, 7), 5), 8), 9)$

What about foldLeft?

For infinite lists

`[7, 1, 2, 3, 4, ...]`

`foldLeft([7, 1, 2, 3, 4, ...], 0, f)` *must* end up as

$\dots f(\dots f(f(f(f(0, 7), 1), 2), 3) \dots) \dots$

What about foldLeft?

For infinite lists

$[7, 1, 2, 3, 4, \dots]$

`foldLeft([7,1,2,3,4,...], 0, f)` *must* end up as

$\dots f(\dots f(f(f(f(0, 7), 1), 2), 3) \dots) \dots$

We do not have an *outermost* f anymore!

What about foldLeft?

For infinite lists

$[7, 1, 2, 3, 4, \dots]$

`foldLeft([7,1,2,3,4,...], 0, f)` *must* end up as

$\dots f(\dots f(f(f(f(0, 7), 1), 2), 3) \dots) \dots$

We do not have an *outermost* f anymore!

No amount of laziness can help us recover!